ABSTRACT

The Inventory Management System is a web-based application designed to simplify product record maintenance, inventory tracking, and stock monitoring. The system is developed using Python as the main programming language, Flask as the web application framework, HTML and CSS for designing the user interface, and SQLite as the database management system. The main objective of this project is to reduce manual work, minimise errors in maintaining product records, and provide accurate and easily accessible inventory information.

The application provides a simple and user-friendly interface through which the administrator can log in securely and access the inventory dashboard. The dashboard displays important information about the products stored in the database and helps the administrator manage inventory activities efficiently. The administrator can add new products, view existing product details, update product information, and delete records when necessary. Each product record may contain details such as product name, category, quantity, price, and product image. These features make product management easier and help maintain organised inventory records.

The system includes product-category pages that allow products to be organised into different groups, such as electronics, groceries, medical items, clothing, furniture, fruits, vegetables, and stationery. Category-wise organisation enables the administrator to locate products quickly and maintain a clear record of available items. A search facility can also be used to find specific products based on their names or other supported details, reducing the time required to retrieve information from the database.

Another important feature of the system is stock-status monitoring. Products can be classified into available, low-stock, and out-of-stock categories based on their quantities. The available-products section helps identify items that are currently in stock, while the low-stock section highlights products that may require replenishment. The out-of-stock section identifies products whose quantities have reached zero. These features help the administrator monitor stock levels, avoid unexpected shortages, and make better decisions regarding inventory maintenance.

The Flask application acts as the connection between the user interface and the SQLite database. When the administrator performs an operation, Flask receives the request, validates the submitted information, executes the appropriate database operation, and returns the required response. SQLite stores the product records in a structured format and supports database operations such as insertion, retrieval, updating, and deletion. HTML and CSS templates present the information in a readable and organised manner, while appropriate status messages help the administrator understand whether an operation has been completed successfully.

INDEX

S.NO.	CONTENT	PAGE NO.
1	INTRODUCTION	7
	1.1. Company Profile	8
	1.2 About the Project	9
2	2. REQUIREMENT SPECIFICATION	10
	2.1 Hardware Specification	10
	2.2 Software Specification	11
	2.3 About the Software	12
3	3. SYSTEM ANALYSIS	13
	3.1 Feasibility System	13
	3.2 Existing System	14
	3.3 Proposed System	15
4	SYSTEM DESIGN	16
	4.1 Normalization	16
	4.2 Table Design / Data Dictionary	17
	4.3 Input Design	18
	4.4 Data Flow Diagram & CFD	19
5	5. SYSTEM DESCRIPTION	20-29
6	6. TESTING AND IMPLEMENTATION	30-32
7	7. CONCLUSION AND FUTURE SCOPE	33-34
8	8. FORMS AND REPORTS	35-40
9	9. BIBLIOGRAPHY	41

1. INTRODUCTION:

The Inventory Management System is a web-based application developed to manage and maintain product information efficiently. Inventory management plays an important role in businesses, shops, warehouses, and other organizations that handle different types of products. Maintaining inventory records manually can be time-consuming and may lead to errors, data duplication, and difficulties in tracking stock availability. To overcome these problems, the Inventory Management System provides a computerized method for storing, updating, viewing, and managing product details.

The main purpose of this project is to simplify inventory-related activities by providing a centralized platform for managing product records. The system is developed using Python as the programming language, Flask as the web framework, HTML and CSS for the user interface, and SQLite as the database. These technologies work together to provide a simple, lightweight, and user-friendly application that can be operated through a web browser.

The system provides an administrator login facility to protect the inventory management functions. After successful login, the administrator can access the dashboard and perform various operations, such as adding new products, viewing existing products, updating product information, and deleting unnecessary records. Product details such as product name, category, quantity, price, and image can be maintained according to the features implemented in the application.

One of the important features of the Inventory Management System is product categorization. Products can be organized into different categories, such as electronics, groceries, medical products, clothing, furniture, fruits, vegetables, and stationery, depending on the categories configured in the application. This classification makes it easier to locate specific products and maintain organized inventory records.

The application also supports stock monitoring by helping the administrator identify products based on their available quantities. Products can be classified into available-stock, low-stock, and out-of-stock groups according to the application's stock-status rules. This feature helps the administrator identify products that require attention and make appropriate decisions regarding stock replenishment. A search facility can also help retrieve product information quickly without manually checking every record.

1.1 Company Profile

Inventory management is important to retail shops, medical stores, stationery outlets and mixed-category businesses. Each organisation needs dependable information about product quantities, selling prices and availability to make daily decisions.

In a small shop, a single administrator may handle purchasing, stock verification, item registration and customer enquiries. A simple computerised record system can support these responsibilities without the expense of a large enterprise platform.

This academic project is not presented as a deployment for a named company. It models the inventory requirements of a typical small or medium-sized store and demonstrates how an administrator can organise records electronically.

Inventory management is important for retail stores, pharmacies, grocery shops, electronics outlets, and other businesses that handle physical products. It involves maintaining records of items, stock quantities, prices, and categories.

Organisations need reliable product records to avoid shortages, reduce unnecessary purchases, and provide timely service. Traditional registers and disconnected spreadsheets may lead to repeated data entry, inconsistent quantities, and delayed information retrieval.

The Inventory Management System in this project is a general-purpose academic application rather than software commissioned by a named company. It demonstrates how a small shop or organisation could maintain inventory records through a web interface.

1.2 About the Project

The application follows a browser-to-server workflow. A user opens an HTML page, the Flask application receives a request, Python performs the required validation or database operation, and a template displays the resulting information.

The principal records are products and administrator credentials. Each product is identified by a unique record identifier, with descriptive fields such as name, category, price, quantity and optional image filename.

The project focuses on the accuracy and accessibility of stock records. It does not automatically represent sales accounting, procurement approval or supplier settlement unless those features are separately implemented.

The Inventory Management System is developed using Python, Flask, HTML, CSS, and SQLite. Flask routes receive browser requests, process form submissions, and render templates. SQLite stores records in a local database file.

The administrator can manage product information through CRUD operations. Product records may include product name, category, price, quantity, and an optional image. Search and category views help locate products quickly.

The project focuses on practical inventory administration using commonly available, open-source technologies.

Objectives of the Project

- Develop a simple, responsive inventory-management interface.
- Maintain product records in a central database.
- Support Create, Read, Update, and Delete operations.
- Allow category-wise product browsing and search.
- Identify available, low-stock, and out-of-stock products.
- Reduce paperwork and improve record retrieval.

Scope and Benefits

The system is appropriate for small-scale inventory demonstration. It can support daily product maintenance and stock review. Benefits include more organised data, easier corrections, faster search, and a foundation for future features.

2. REQUIREMENT SPECIFICATION

The requirement specification describes the hardware, software, functional, and non-functional requirements necessary for developing and operating the Inventory Management System. It defines the resources needed to run the application and the features that the system must provide to manage inventory records efficiently. The system is developed using Python, Flask, HTML, CSS, and SQLite.

2.1 Hardware Specification

A conventional laptop or desktop is sufficient for local development and demonstration. Actual resource usage depends on the number of stored products, the size of uploaded images and the browser and editor running at the same time.

A stable storage device is especially important because the SQLite database and uploaded image files are saved locally. Regular backups should include both the database and the uploads folder.

The project can be developed and demonstrated on a standard laptop or desktop computer. The suggested configuration is shown below.

Hardware Component	Suggested Specification
Processor	Dual-core processor or better
RAM	4 GB minimum; 8 GB recommended
Storage	At least 1 GB free space for application and database
Monitor	1366 × 768 resolution or higher
Keyboard and Mouse	Standard input devices
Network	Localhost requires no internet; internet for dependency installation

The processor runs the Flask application and browser. RAM supports the development environment and database access. Storage is used for source code, the SQLite database, and optional uploaded product images.

2.2 Software Specification

The Python runtime executes the Flask application. Visual Studio Code can be used to edit Python, HTML and CSS files and to run commands in an integrated terminal.

The browser sends HTTP requests to Flask and renders the returned pages. SQLite provides an embedded database engine, avoiding the need for a separately managed database server in a local demonstration.

Dependencies should be installed in a virtual environment and recorded in requirements.txt to make setup more repeatable on another computer.

Software	Purpose
Windows / Linux / macOS	Operating system
Python 3	Backend programming
Flask	Web framework and routing
SQLite	Relational database
HTML and CSS	Frontend pages and design
Visual Studio Code	Code editing and debugging
Web Browser	Accessing the web application
Werkzeug / SQLAlchemy	Utilities and database ORM, if used

2.3 About the Software

Flask maps URL paths to Python view functions. These functions may query the database, process submitted forms, perform redirects and render HTML templates with current product data.

HTML provides semantic structure for headings, forms and tables. CSS controls visual hierarchy, responsive layouts, spacing and colour. SQLite stores structured records and supports SQL operations for inserting, selecting, updating and deleting rows.

If Flask-SQLAlchemy is used, model classes represent database tables and the ORM translates model operations into SQL. If direct sqlite3 queries are used instead, the same database principles apply.

Python is a general-purpose programming language used for backend logic and validation. Flask is a lightweight Python web framework that maps URLs to functions, processes requests, and renders HTML templates.

SQLite is a file-based relational database that supports SQL queries without requiring a separate database server. HTML defines page structure; CSS controls layout, colours, and responsiveness.

SQL commands include INSERT for new records, SELECT for retrieval, UPDATE for modifications, and DELETE for removal. SQLAlchemy may be used to manage database tables through Python models.

3. SYSTEM ANALYSIS

System Analysis is an important phase in the development of the Inventory Management System. It involves examining the existing inventory management process, identifying its limitations, understanding user requirements, and defining the features needed for the proposed application. The main objective of system analysis is to develop a reliable and user-friendly application that simplifies product management, improves record maintenance, and supports efficient stock monitoring.

The Inventory Management System is designed to overcome common difficulties associated with manual inventory management. In a manual system, product details may be maintained in notebooks, registers, or spreadsheets, making it difficult to search for specific items, update quantities, and identify products that are running out of stock. The proposed system provides a centralized web-based application for managing inventory information.

3.1 Feasibility System

Technical feasibility: the project uses widely available tools with extensive documentation and modest hardware requirements. Flask and SQLite are appropriate for learning server-side development and relational data storage.

Economic feasibility: the software stack is open source and can be developed on an existing computer. Costs mainly involve development time, maintenance and backup storage.

Operational feasibility: the interface is based on familiar forms and product lists. A new administrator can be trained to add, update and locate records with limited instruction.

Schedule feasibility: development can be divided into requirements, database design, interface design, CRUD implementation, validation, testing and documentation.

Technical feasibility: Python, Flask, and SQLite are suitable for a small web application, and development can be performed on a standard computer.

Economic feasibility: The selected technologies are free to use, keeping software costs low. Development time, maintenance, and backup effort are the main considerations.

Operational feasibility: Familiar forms, tables, category navigation, and clear messages can help an administrator maintain records with limited training.

Schedule feasibility: Development can be divided into requirements, database design, interface development, backend coding, integration, testing, and documentation.

Security feasibility: Administrative functions require access control, safe password storage, input validation, and regular backups.

3.2 Existing System

In a manual inventory system, stock information may be maintained in notebooks or individual spreadsheet files. Staff must compare entries to find current quantities, and corrections can be difficult to trace.

When stock is received or issued, delayed record updates can cause discrepancies between recorded and physical quantities. Category-wise searching and identifying low-stock items may also require repeated manual checking.

These observations describe common limitations of manual workflows rather than measured results from a particular shop.

In a manual inventory system, product names, prices, and quantities are recorded in registers or spreadsheets. Staff must locate the appropriate record and change the stock count when items are added or removed.

Limitations include time-consuming search, duplicate records, inaccurate stock balances, difficulty locating low-stock items, inconsistent category naming, and possible loss of records.

The existing inventory management system in many small shops and organisations relies on manual methods for maintaining product information and monitoring stock levels. Product details such as product names, prices, quantities, categories, and available stock are commonly recorded in notebooks, registers, or individual spreadsheet files. These records must be updated whenever new products are purchased, existing products are sold, or damaged items are removed from stock. Although this method may be suitable for managing a small number of products, it becomes difficult to maintain when the number of products and daily transactions increases.

In a manual inventory system, staff members must search through registers or spreadsheet files to identify a particular product and determine its current quantity. This process can consume considerable time, especially when products are arranged across multiple categories. If the records are not organised properly, locating the required information becomes more difficult. Maintaining separate records for electronics, groceries, medical items, clothing, furniture, fruits, vegetables, and stationery may also result in inconsistent product classification.

3.3 Proposed System

The proposed system maintains product information in a central SQLite database. Administrators interact with it through web forms and product lists instead of editing database files directly.

An add-product operation validates submitted fields before saving a record. An edit operation loads an existing product and updates selected fields. A delete operation removes a record after the appropriate confirmation.

Category filtering, search and stock-status views help the administrator locate records. The interface should display useful feedback when a product is missing or input is invalid.

The proposed application centralises product information in an SQLite database and provides a browser interface for administrative tasks. The administrator can add new products, view and edit records, delete obsolete entries, search by relevant terms, and browse categories.

Separate stock-status pages support routine monitoring. The system is designed to reduce repeated entry and simplify record maintenance, although its reliability still depends on accurate data input and testing.

Advantages of the Proposed System

- Centralised and structured product records.
- Simplified product CRUD operations.
- Faster category-wise browsing and search.
- Improved visibility of stock status.
- Accessible interface using standard web technologies.
- Low-cost academic implementation.

4. SYSTEM DESIGN:

System Design is an important phase in the development of the Inventory Management System. It describes the overall structure of the application, the interaction between its components, the flow of information, and the methods used to store and manage inventory records. A well-designed system helps ensure that the application is organized, reliable, easy to maintain, and convenient for the administrator to use.

The Inventory Management System is designed as a web-based application using Python, Flask, HTML, CSS, and SQLite. Flask handles application logic and user requests, HTML provides the structure of the web pages, CSS controls the appearance of the interface, and SQLite stores product information and other required records.

4.1 Normalization

First Normal Form requires atomic field values and uniquely identifiable rows. A product record should store a single product name and a single quantity value rather than a list of several items in one field.

Second Normal Form removes partial dependencies on a composite key. With a simple product_id primary key, descriptive product fields depend on that identifier.

Third Normal Form avoids storing unrelated attributes that depend on other non-key attributes. For a larger implementation, category descriptions may be stored in a separate category table referenced by category_id.

The actual schema should be reviewed before claiming that every normal form has been achieved. A simple academic design may intentionally keep category names directly in the product table.

Database normalization reduces unnecessary duplication and helps keep related information consistent. First Normal Form requires atomic field values; Second Normal Form removes partial dependencies on composite keys; Third Normal Form removes transitive dependencies.

A basic implementation may store products and administrators in separate tables. A more advanced design can place categories in their own table and reference them from products. The final database structure must match the implemented models.

4.2 Table Design / Data Dictionary

The data dictionary documents the meaning, type and purpose of stored fields. It supports implementation, maintenance and testing by providing a shared definition of each record.

Typical product fields include product_id, name, category, quantity, price and image_filename. Constraints should reject negative quantities and invalid prices where supported by the implementation.

An administrator record commonly contains a unique identifier, username and a password hash. A password hash is preferable to storing the original password.

The following tables show a suggested data dictionary for a Flask and SQLite inventory application. Actual column names and data types should be checked against models.py.

Product Table

Field	Type	Description
id	INTEGER PRIMARY KEY	Unique product identifier
name	TEXT	Product name
category	TEXT	Product category
quantity	INTEGER	Available units
price	REAL / NUMERIC	Unit price
image	TEXT (optional)	Stored image filename

Administrator Table

Field	Type	Description
id	INTEGER PRIMARY KEY	Unique admin identifier
username	TEXT UNIQUE	Administrator username
password_hash	TEXT	Securely hashed password (recommended)

4.3 Input Design

Input forms should clearly label every field and indicate which values are required. Numeric fields such as price and quantity should be checked for valid ranges before database insertion or update.

The add-product form collects product details, while the edit form displays saved values for revision. Search forms accept a keyword and return matching items or a clear no-results message.

Uploaded image files should be limited to supported file types, assigned safe filenames and stored in the configured upload directory.

The login form accepts a username and password. The add-product form collects product name, category, price, quantity, and optionally an image. The edit-product form displays existing values for correction.

Validation should reject missing names, negative quantities, invalid prices, and unsupported image formats. Forms should provide useful error or success messages.

4.4 Data Flow Diagram & CFD

At context level, the administrator sends login details, product information and search requests to the Inventory Management System. The system reads or updates the product database and returns status messages or product lists.

At level one, the major processes are authentication, product maintenance, category browsing, search and stock monitoring. Product maintenance exchanges data with the product store; authentication checks administrator information.

A control flow begins with opening the application, logging in, choosing an action, validating input, accessing the database, displaying the result and finally logging out.

The administrator sends login credentials or product information to Flask. Flask validates the request, communicates with SQLite, and returns an HTML page containing the result. Search and stock-status pages read product records without changing them.

Inventory Management System – Data Flow

ADMINISTRATOR

Login / Product Details / Search Request

FLASK APPLICATION

Validate and Process Request

SQLITE DATABASE

Store and Retrieve Product Records

FLASK HTML TEMPLATES

Dashboard / Product Lists / Status Messages

Control flow: Start → Login → Validate Credentials → Dashboard → Select Operation → Add / View / Edit / Delete / Search / Stock Status → Database Operation → Display Result → Logout.

5. SYSTEM DESCRIPTION

The Inventory Management System is a web-based application developed using Python, Flask, HTML, CSS, and SQLite. The main purpose of this system is to simplify the process of managing product information, maintaining inventory records, and monitoring stock availability. It provides a centralized platform where the administrator can add new products, view existing products, update product details, delete unwanted records, and organize products into different categories.

The system replaces many of the difficulties associated with manual inventory management, such as maintaining handwritten records, searching for products manually, calculating stock quantities, and identifying products that require replenishment. By storing product information in a database, the application makes inventory records easier to access, update, and maintain.

The application provides an administrator login module that restricts access to administrative functions. After successful authentication, the administrator can access the dashboard, which acts as the main navigation area of the system. The dashboard may display inventory summaries, including the total number of products, available products, low-stock products, and out-of-stock products, depending on the implemented functionality. These details help the administrator understand the current stock situation.

The Product Management Module supports the four basic database operations: Create, Read, Update, and Delete (CRUD). When adding a product, the administrator can enter details such as product name, category, price, quantity, and image. The application validates the submitted information before storing it in the SQLite database. Existing records can be viewed and modified, while unnecessary product records can be deleted by an authorized administrator.

Login Module:

The login page collects a username and password. After successful verification, the application creates an authenticated session that allows access to administrative routes.

An invalid login should display a neutral error message without exposing whether a username exists. Session protection and secure password hashing are important for a real deployment.

The administrator enters credentials through a login page. Flask validates the submitted information and establishes a session when authentication succeeds. Incorrect credentials should be rejected. Passwords should be stored as hashes.

Password Security

Passwords must be handled securely to protect administrator accounts. Instead of storing passwords as plain text, the application should store password hashes generated using a trusted password-hashing method. During login, Flask verifies the entered password against the stored hash.

A secure secret key should be configured for session protection, and appropriate cookie settings should be used in a deployed application. For real-world use, HTTPS should also be enabled to protect credentials while they are transmitted between the browser and the server.

Form Validation and Error Handling

The login module should validate the submitted information before attempting authentication. The username and password fields should not be empty, and unexpected input should be handled safely. If the form contains missing or invalid information, the application should display a suitable message and allow the administrator to try again.

The system should also handle authentication errors without exposing internal application details, database information, or technical error messages. These checks improve usability and help maintain the security of the application.

Dashboard Module:

The dashboard serves as the main navigation area for the administrator. It may show summary counts for products and stock conditions, along with links to add, edit and browse products.

Dashboard totals should be calculated from current database records so that the displayed values stay consistent with product changes.

The dashboard provides navigation to product management, category pages, and stock-status views. Depending on the implementation, it may display counts for total, available, low-stock, and out-of-stock products.

Dashboard Overview

The dashboard displays a summary of the current inventory status. Depending on the implementation, it may show the total number of products, available products, low-stock products, and out-of-stock products. These summary values help the administrator understand the current condition of the inventory without opening each product record individually.

The dashboard may also display welcome messages, navigation menus, summary cards, and links to frequently used functions. A well-organized layout makes the application easier to understand and improves the overall user experience.

Product Management Navigation

The dashboard provides navigation links to the main product management functions. The administrator can use these links to add new products, view existing products, edit product details, and delete unwanted records.

Product Management Module:

The create operation inserts a new product after validation. The read operation retrieves products for display. The update operation modifies an existing record, and the delete operation removes a selected record.

A product should be identified using its unique ID rather than relying on the name alone, because two products may have similar names.

The CRUD module allows the administrator to create product records, read product details, update information, and delete records. Product fields may include name, category, price, quantity, and image.

Create Product

The Create operation allows the administrator to add new products to the inventory database. The administrator enters the required product details through an input form, including product name, category, price, quantity, and product image, depending on the fields supported by the application.

Read and View Product Details

The Read operation retrieves product records from the SQLite database and displays them on the product listing page. The administrator can view the available information for each product, such as its unique ID, name, category, price, quantity, and image.

Update Product Information

The Update operation allows the administrator to modify the details of an existing product whenever changes occur. For example, the administrator may need to update the product name, category, price, quantity, or image.

Delete Product

The Delete operation allows the administrator to remove a product record that is no longer required. The administrator selects the appropriate product and activates the Delete option. The application identifies the selected record using its unique ID and removes it from the database after the request is authorized.

Category Module:

Category pages group related items so an administrator can browse a smaller selection. Example categories include groceries, fruits, electronics, medical items, clothing, furniture and stationery.

The category field should use consistent values so that products appear under the correct headings.

Products are organised into categories including electronics, groceries, medical items, clothing, furniture, fruits, and stationery. Separate pages make category-wise browsing convenient.

Product Categorization

Product categorization involves assigning each product to a specific group according to its nature. For example, laptops, keyboards, and computer accessories can be placed under Electronics, while notebooks, pens, and pencils can be placed under Stationery.

Separate Category Pages

The system can provide separate pages for each product category. When the administrator selects a category, the application retrieves the products associated with that category and displays them on the corresponding page.

Category-Based Database Retrieval

The Category Module interacts with the SQLite database through the Flask application. Each product record contains a category field that identifies the group to which the product belongs.

Consistent Category Values

Maintaining consistent category names is essential for accurate product classification. The application should use standardized category values when products are added or updated. For example, values such as `Electronics`, `electronics`, and `Electronic Items` should not be used interchangeably if the application treats them as different categories.

Search Module

The search module accepts a keyword and checks relevant product fields. Results are displayed in a table or card layout with links to available actions.

Search should handle empty keywords and no-match cases gracefully. Parameterised queries or ORM filters help prevent unsafe query construction.

A search term is submitted through a form and compared against supported product fields. Matching products are displayed, and a clear message should appear when no results are found.

Keyword-Based Search

The Search Module allows the administrator to search for products using relevant keywords. Depending on the implementation, the search may check fields such as product name, category, or other supported product details.

Database Integration

The Search Module retrieves information directly from the SQLite database through the Flask application. When a search request is submitted, the application executes an appropriate database query to identify matching records.

Displaying Search Results

After processing the search request, the application displays the matching products in an organized format. Each result may include product details such as product ID, name, category, price, quantity, and image, depending on the fields supported by the system.

Stock Status Module:

Available stock generally means quantity greater than zero. Out-of-stock items have quantity equal to zero. A low-stock threshold can identify products requiring replenishment.

The report must state the exact threshold used by the finished application. Different product types may require different reorder limits in a future version.

Available-products pages display items with stock on hand. Low-stock pages identify products at or below the configured threshold. Out-of-stock pages display products with zero quantity.

Available Stock Management

Available stock refers to products that have a quantity greater than zero. These products are currently available in the inventory and can be used or sold, depending on the purpose of the system.

Low-Stock Monitoring

The Low-Stock Module identifies products whose quantities have reached a predefined minimum stock level. These products may still be available, but their quantities are low enough to require attention from the administrator.

Out-of-Stock Monitoring

Out-of-stock products are items whose recorded quantity is equal to zero. These products are currently unavailable in the inventory and may require immediate replenishment.

Image Upload Module:

Product photographs improve visual recognition, especially when items have similar names. The database typically stores a relative filename rather than the binary image itself.

A safe implementation validates the file extension, limits upload size and handles missing image files with a placeholder or descriptive text.

When implemented, image uploads allow a product picture to be stored and displayed. Secure filenames, permitted extensions, and fallback images help avoid errors.

Uploading Product Images

The Image Upload Module allows the administrator to associate a photograph with a product record. The Add Product or Edit Product form can include a file selection field through which an image is selected from the computer.

Image Storage and Database Integration

Uploaded product images are generally stored as separate files rather than being saved directly as binary data inside the SQLite database. The database stores the image filename or relative path associated with the product record.

Database Connectivity:

Flask application logic communicates with SQLite using a database library or an ORM. Queries retrieve product rows and transactions save new or modified data.

The database file should be backed up regularly, and write operations should handle exceptions so that failed requests do not leave the interface in an inconsistent state.

Flask accesses SQLite directly or through an ORM such as SQLAlchemy. The database stores product records and administrator details. Transactions should be committed after successful updates, with errors handled appropriately.

Database Connection Establishment

When the Flask application starts or a database operation is requested, the application establishes access to the SQLite database file. The database connection allows the application to execute SQL queries and retrieve the required information.

Database Operations

The application performs different database operations according to the administrator's actions. These operations support the main functions of the Inventory Management System.

- Insert operation: Adds new product records to the database.
- Select operation: Retrieves product information for display on the dashboard, product listing, and category pages.
- Update operation: Modifies existing product details, such as price, quantity, category, or image filename.
- Delete operation: Removes selected product records from the database.

Security and Backup

Administrator routes should require a valid session. User input must be validated on the server even when browser-side checks are present.

Backups should preserve the SQLite database and any uploaded product images. A recovery test verifies that saved files can be restored successfully.

Administrative routes should be protected. Forms require validation, passwords should be hashed, and uploaded files should be checked. Regular copies of the database and upload folder are important for recovery.

Input Validation and Data Protection

All user inputs must be validated on the server before they are stored in the SQLite database. Although HTML forms may include browser-side validation, server-side validation is essential because browser checks can be bypassed.

The application should verify that product names are not empty, quantities are valid numbers, prices are non-negative, and category values are valid. Appropriate error messages should be displayed when invalid information is entered. These checks help prevent incorrect records, unexpected application errors, and inconsistent inventory data.

Database queries should use parameterized statements instead of directly combining user input with SQL commands. This reduces the risk of SQL injection and helps protect the database from malicious input.

Password Security

Administrator passwords should never be stored as plain text in the database. Instead, passwords should be stored using a secure password-hashing method provided by a trusted library. During login, the application verifies the entered password against the stored hash.

The application should also use a secure session configuration and a strong secret key to protect session data. In a deployed application, HTTPS should be enabled to protect login credentials and other sensitive information during transmission. These measures improve the overall security of administrator access.

6. TESTING AND IMPLEMENTATION

Software Testing

Unit tests can check validation rules and helper functions. Integration tests can verify that form submissions update the database and that product lists reflect the saved changes.

System testing examines the full workflow, including login, add product, edit product, search, stock views and logout. Negative tests check missing fields, invalid quantities and nonexistent product IDs.

Test scenarios in this report are proposed checks; their outcomes must be recorded from the actual running application before being marked as passed.

Testing verifies that the application works as expected and handles invalid inputs safely. Unit testing checks individual functions; integration testing checks routes, templates, and database interaction; system testing checks the full workflow; validation testing checks incorrect or incomplete data.

Test ID	Test Scenario	Expected Result
TC01	Valid administrator login	Dashboard opens
TC02	Incorrect login password	Access denied
TC03	Add valid product	Product saved
TC04	Submit blank product name	Validation error
TC05	Edit product price	Updated value displayed
TC06	Delete a test product	Record removed
TC07	Search existing product	Matching item displayed
TC08	Search unknown product	No-results message
TC09	Set quantity to zero	Out-of-stock classification
TC10	Upload unsupported file	File rejected, if upload enabled

These are proposed test cases; actual pass/fail results should be recorded after executing the final application.

Implementation

The developer creates a virtual environment, installs the required Python packages, configures the SQLite database and prepares the templates and static folders.

After the application starts, the developer opens the local Flask URL in a browser and tests each route with sample product records.

For final demonstration, the project folder should include source files, requirements, database setup instructions and a clear explanation of how to launch the server.

- Install Python and create a project virtual environment.
- Install Flask and the database libraries used by the application.
- Create app.py, models.py, templates, and static directories.
- Configure SQLite and create the required database tables.
- Implement login, product CRUD, category, and stock-status routes.
- Run the application locally and test all forms and pages.
- Back up the database and capture screenshots of working output.

Maintenance

Maintenance includes fixing errors, improving validation, updating dependencies, adjusting category lists and refining the interface based on user feedback.

Data maintenance includes correcting incorrect product entries, reviewing low-stock thresholds and verifying that backups remain usable.

Maintenance includes fixing defects, updating dependencies, reviewing validation, making database backups, and testing new features before deployment.

1. Error Detection and Correction

Maintenance involves identifying and correcting errors that may occur during application execution. These errors may include incorrect product details, problems with login authentication, failures during product addition or updating, database connection issues, and errors in displaying product lists. Each error should be examined carefully, and appropriate corrections should be made to prevent the same problem from occurring again.

2. Input Validation and Data Accuracy

The application should be checked regularly to ensure that product information is entered correctly. Validation rules should prevent missing product names, invalid prices, negative quantities, and unsupported image files where applicable. Incorrect product records should be reviewed and corrected to maintain the accuracy and consistency of the inventory database.

3. Database Maintenance

The SQLite database stores important information, including product names, categories, prices, quantities, and image filenames. Database maintenance includes checking stored records, removing unnecessary duplicate entries, correcting inaccurate information, and verifying that database operations work properly. Regular database backups should be maintained to reduce the risk of data loss caused by accidental deletion, software errors, or system failures.

4. Software and Dependency Updates

The Python environment, Flask framework, and other installed libraries should be reviewed periodically. Updating dependencies when appropriate can improve compatibility, reliability, and security. Before applying an update, the application should be tested to ensure that existing features continue to work correctly. Unnecessary or unused packages should also be reviewed to keep the project environment organised.

7. CONCLUSION AND FUTURE SCOPE

Conclusion

The project demonstrates how web forms, Python application logic and a relational database can work together to manage product records.

Its main value lies in making inventory information easier to maintain, search and review through a single interface. The application remains a small-scale academic solution and should be tested and secured before practical deployment.

The Inventory Management System demonstrates a structured approach to maintaining product records using Python, Flask, HTML, CSS, and SQLite. The system brings together administrator access, product CRUD operations, category browsing, search, and stock-status views in a web application.

The project provides practical experience in web development, database design, request handling, input validation, and testing. A properly tested implementation can make routine inventory record maintenance easier and more consistent.

The system brings together important inventory management functions, including administrator login, dashboard access, product creation, product viewing, updating, and deletion. It also supports category-wise product browsing, search functionality, and stock-status monitoring. Product details such as name, category, price, quantity, and image can be maintained in a structured manner, depending on the features implemented in the application. These functions help the administrator access the required information more conveniently and maintain organised product records.

One of the major benefits of the proposed system is the centralised storage of product information in an SQLite database. Instead of depending entirely on paper records or separate spreadsheet files, the administrator can manage product details through a single web interface. Category pages make it easier to locate related products, while available-stock, low-stock, and out-of-stock views help identify the current status of inventory. These features can reduce repetitive manual work and support better stock monitoring when product information is entered and updated correctly.

Future Scope

Possible future enhancements include purchase and sales transactions, supplier records, barcode scanning, exportable reports, role-based access, automated low-stock notifications and audit history.

A future version could support multi-user access, stronger deployment security and more advanced stock analytics once the basic workflow is reliable.

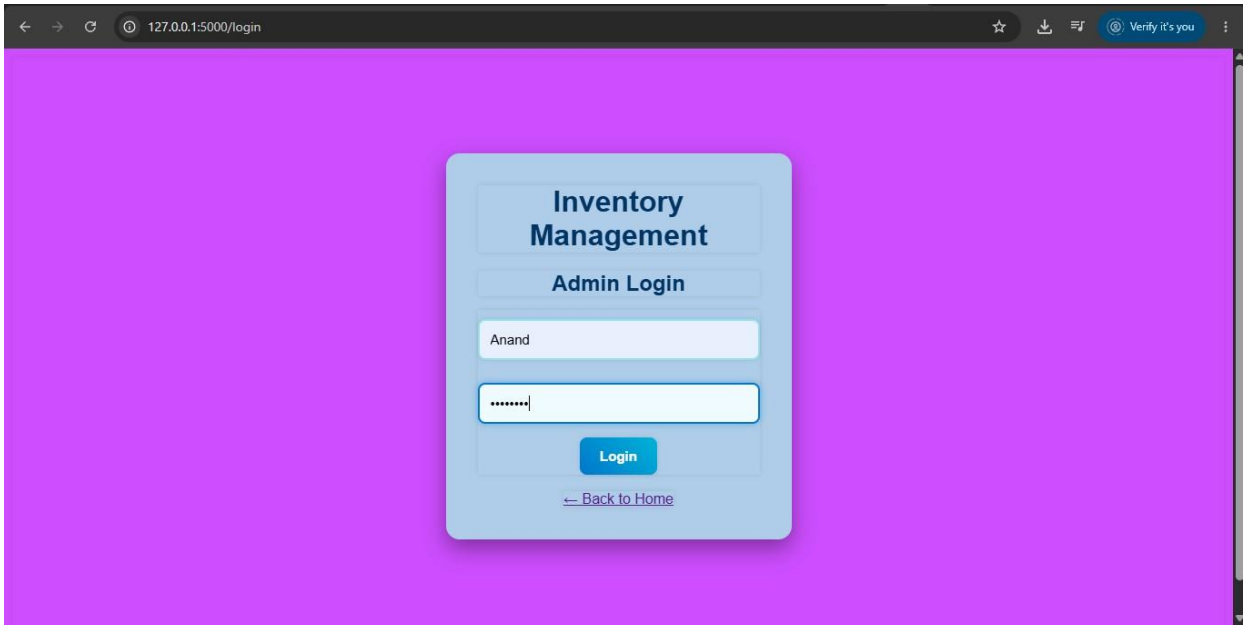
The application can be extended with supplier records, purchase and sales transactions, reorder thresholds, barcode or QR-code scanning, printable reports, CSV or Excel export, role-based access, activity logs, and automated low-stock alerts. Cloud deployment and mobile-friendly improvements can also be considered after security testing.

8. FORMS AND REPORTS

Administrator Login Page

The login form contains username and password inputs and a submit action. The application checks the credentials and either opens the dashboard or displays an error.

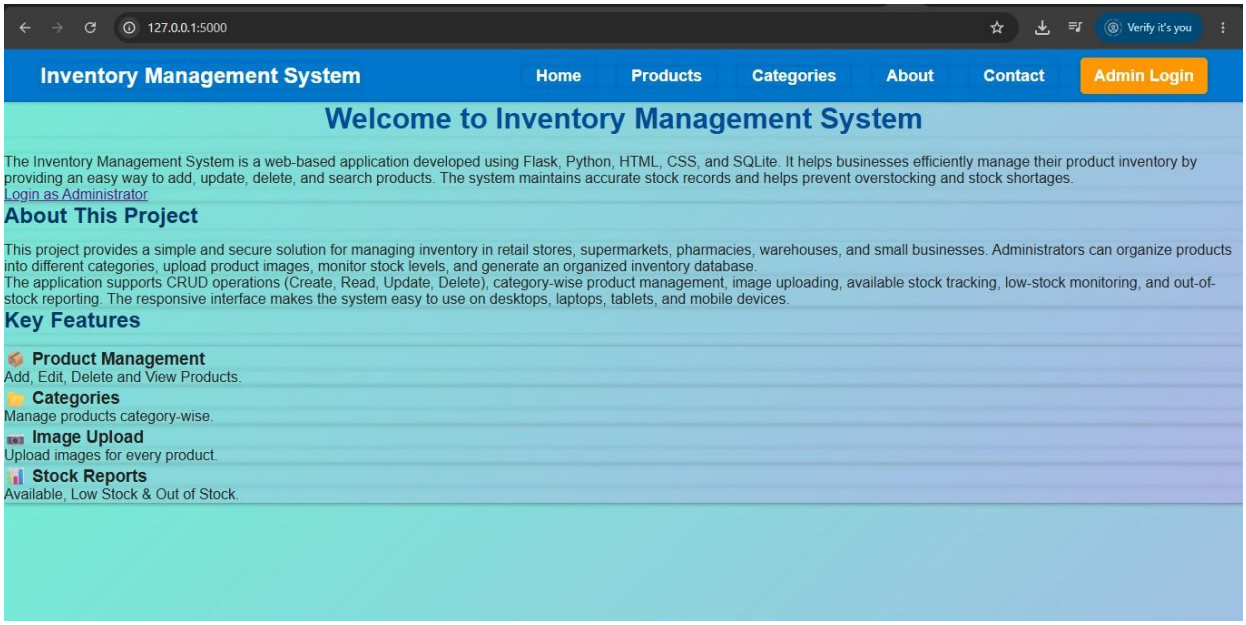
The login screen collects administrator credentials and grants access to protected management pages after successful authentication.



Home Page

The home page introduces the system and provides navigation to available product and category pages. Its layout should remain readable on common screen sizes.

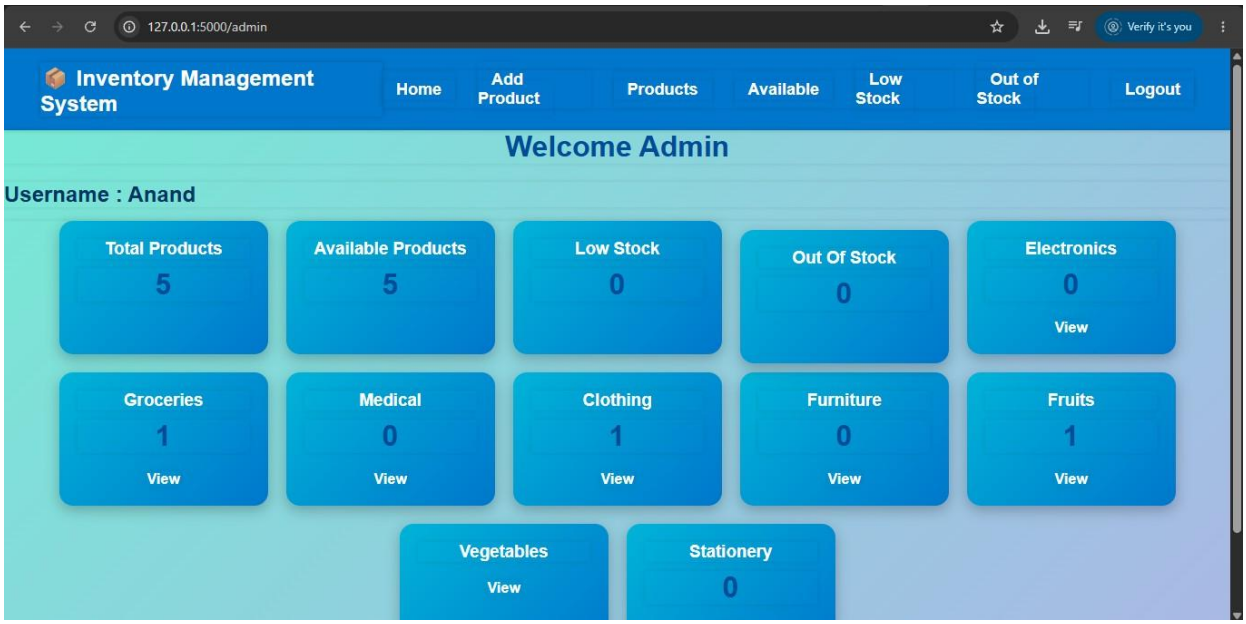
The home page introduces the Inventory Management System and provides navigation to the main application sections.



Dashboard

The dashboard provides administrative links and, where implemented, summary indicators calculated from the product records.

The dashboard serves as a central page for product management and stock overview.



Add Product Form

The add-product form captures product name, category, quantity, price and optional image information. Validated values are saved to the database.

The add-product form records a new product name, category, price, quantity, and optional image.



Product List and Edit Form

The product list shows stored records and actions for editing or deleting a selected item. The edit form allows permitted fields to be changed.

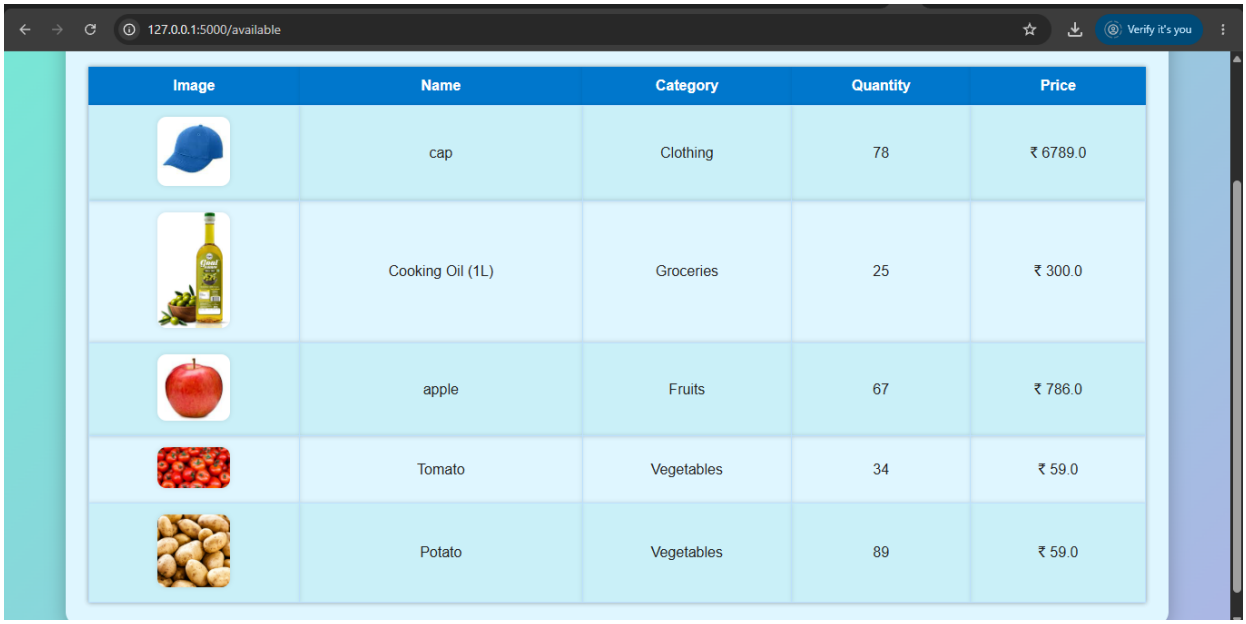
The product list displays stored records and allows an authorised administrator to open details, edit information, or delete a record.



Category Pages

Category pages display products grouped by type, supporting faster browsing of a large product collection.

Category views display related products such as electronics, groceries, medical items, clothing, furniture, fruits, and stationery.



9. BIBLIOGRAPHY

- Python Software Foundation. Python Documentation. https://docs.python.org/3/
- Pallets. Flask Documentation. https://flask.palletsprojects.com
- SQLite. SQLite Documentation. https://www.sqlite.org/docs.html.com
- MDN Web Docs. HTML Documentation. https://developer.mozilla.org/en-US/docs/Web/HTML.com
- MDN Web Docs. CSS Documentation. https://developer.mozilla.org/en-US/docs/Web/CSS.com
- SQLAlchemy Documentation. https://docs.sqlalchemy.org/ (if used)
- Pressman, Roger S. and Bruce R. Maxim. Software Engineering: A Practitioner's Approach. McGraw-Hill.